

Document Number: P2766R0

Date: 2023-01-14

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2022-10-12 through 2022-12-14

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [October 12th, 2022](#)
- [October 19th, 2022](#)
- [November 2nd, 2022](#)
- [November 30th, 2022](#)
- [December 14th, 2022](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)
- [P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26](#)
- [P2512: SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15](#)
- [P2605: SG16: Unicode meeting summaries 2022-01-12 through 2022-06-08](#)
- [P2678: SG16: Unicode meeting summaries 2022-06-22 through 2022-09-28](#)

October 12th, 2022

Draft agenda:

- Michael Kuperstein: Internationalization From the Perspective of Defect Analysis
- NB comment processing.

Attendees:

- Charles Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Mark Zeren
- Michael Kuperstein
- Nevin Liber
- Peter Brett
- Steve Downey
- Tom Honermann
- Tomasz Kamiński
- Victor Zverovich

Meeting summary:

- Michael Kuperstein: Internationalization From the Perspective of Defect Analysis
 - [Editor's note: Michael's slides are available at <https://github.com/sg16-unicode/sg16-meetings/blob/master/presentations/2022-10-12-i18n-presentation.pptx>.]
 - Michael provided a brief introduction:
 - He has been working for Intel since 1996.
 - He has been working in Intel's localization group since 2001.
 - Slide 1: Internationalization From the Perspective of Defect Analysis
 - Slide 2: Venn Diagram
 - Slide 3: Defects in Localized Software
 - The defect breakdown presented is from an analysis performed in 2011.
 - Internationalization and localization defects are usually found by the localization team.
 - Localization defects can often be fixed by the localization team; as a result, localization teams tend to maintain their own defect database.
 - Localization defects that require a fix by a development team tend to first be reported in a defect database maintained by the localization team and then migrated to another team's defect database.
 - Slide 4: World-Readiness Defect Types
 - Most localization defects are due to UI, Layout, or formatting issues.
 - The next largest category of defects are due to translation issues.

- Defects due to non-translated and embedded strings make up the next largest two categories.
- Defects due to encoding issues make up the smallest defect category, but are very important.
- For software developers, internationalization and localization support is a small part of their total effort, but an important part.
- Slide 5: Code Scans: I18N Issues by Volume
 - The top two categories of issues found by code scans are hard-coded strings and hard-coded formatting.
- Slide 6: I18N Issues by Volume – Honorable Mentions
 - A consistent internal locale insensitive representation of dates is necessary to prevent failures.
 - Steve confirmed that the general shape of relative error counts presented matches his experience.
 - Steve reported that products he has worked on avoid localized formatting of dates so as to avoid confusion; likewise, "." is consistently used for decimal point.
- Slide 7: More than 150 string formatting functions in C/C++ on Windows
 - Charlie noted that most of those 150 functions wrap a common underlying formatting function.
 - Corentin suggested bumping the number to 151 now that `std::format()` has been standardized.
- Slide 8: Defaults: Fall into the pit of success
 - Use of UTF-16 made it easier to produce the right results on Windows.
 - A string class that basically does the right thing makes it easier to get the right result.
 - The goal is to guide developers towards doing the right thing.
 - Many programmers like string interpolation.
 - ICU discussion:
 - Charlie reported that the ICU included in Windows doesn't expose the C++ interface.
 - Michael noted that, in .NET languages, programmers can choose either ICU or the native Windows NLS subsystem for localization, but programmers generally use the default.
 - Charlie asked if ICU is mostly present for transcoding purposes.
 - Michael replied that he doesn't believe that to be the case since .NET interfaces can defer to ICU for more localization purposes.
 - Michael expressed a belief that ICU is more deeply integrated on Apple systems.
 - PBrett asked what defect category would best be associated with cases where programmers incorrectly attempt to produce translated strings via concatenation.

- Michael expressed uncertainty, suggested "other", and noted that such issues are very common but not called out specifically in the slides.
 - Michael acknowledged that, for some applications, issues due to concatenation are one of the most common problems, but that doesn't happen to be the case for Intel.
 - Michael reiterated that making sure programmers fall into the pit of success is important.
- Slide 9: Quick Intro to BCP 47 Language Tags and Fallback
 - Spoken language is not relevant for text presentation; written language, or script, is.
 - Chinese has two forms of written language; simplified and traditional.
 - It is important to specify fallback locales; otherwise, a request for zh-SG when it is not available may result in a default language like English rather than zh-CN.
 - Specifying a hierarchy of fallbacks such as zh-Hans and zh-Hant is recommended.
 - Since C++ locales don't appear to provide locale fallbacks, it may be necessary to supply support for all of them; perhaps by providing the same locale data for, e.g., zh-CN and zh-SG.
 - Steve noted that English is a better fall back than blank strings or the "tofu" character.
- Slide 10: User Language Selection Choices
 - The .NET languages wrap locale info in a `CultureInfo` type.
 - They also allow various components of a locale to be selected from different locales.
 - Programmers can create their own custom cultural definitions.
 - Thread specific locale selection is infrequently used; it is more common to supply a locale object locally when constructing a string for presentation.
 - Browsers have multiple language settings; one for the browser UI itself and another for the requested page language.
- Slide 11: Date formatting
 - Use ISO 8601 for date formatting and store times relative to UTC internally.
 - Convert dates to the appropriate locale for presentation.
 - Likewise, use one encoding internally and convert for presentation and at program boundaries.
 - Hubert asked if Michael had any opinions on the use of ISO week days and numbers.
 - Michael responded that he has no opinion on that.
- Slide 12: The Famous Turkish "i" Problem
 - Locale sensitive uppercasing may translate "i" to "İ" (dot retained on uppercase I).
 - Locale sensitive lowercasing may translate "I" to "ı" (dot omitted on lowercase i).

- This is why it is important to test with Turkish locales!
- Various languages offer locale invariant or case insensitive case folding operations.
- ICU collation solves many of these problems when used correctly.
- Some form of collation should be used for file name matching.
- Hubert asked if it would generally be expected for a file with an uppercase dotted I like "FILE.GiF" to match a request for files named with a ".gif" extension.
- Michael responded affirmatively; that would generally be desired.
- Tom observed that such use cases may be more aligned with a form of transliteration.
- Corentin responded that Unicode case folding as defined in [UAX #35](#) handles that case, but that standard C++ doesn't provide an interface.
- Slide 13: Formats (numbers, dates, etc.) are not as straightforward as they appear
 - ICU's message formatting abilities handle all of these.
 - Corentin noted that currency symbols should not be locale dependent and that C++ got this wrong.
- Slide 14: Many other things can go wrong when dealing with international users
 - Handling plural forms is important; the .NET languages do not handle plural forms or gendering.
- Slide 15: JavaScript i18n Objects and Namespaces
 - JavaScript only provides a small number of builtins; i18n is a separate package.
 - Current browser versions provide the JavaScript i18n namespace; polyfill is required for older browser versions.
 - Since the language doesn't provide it as a builtin, there are thousands of i18n packages available.
- Slide 16: .NET Culture Aware Classes and Namespaces
 - The .NET languages provide a relatively complete solution that is improving each year.
 - The .NET fundamentals documentation is extensive.
 - Resource files are easy for .NET languages and can be provided in a number of formats.
 - The .NET languages support gettext-like methods for retrieving translated strings.
- Slide 17: Resource File Formats
 - Some resource file formats are differentiated by encoding.
- Slide 18: Read All Lines From a File
 - Some languages provide more ergonomic interfaces.
- Slide 19: Byte Order Mark (BOM) and Endian descriptions
 - On Windows, the default encoding used to be a locale dependent "ANSI" encoding, but modern editors are more likely to default to UTF-8.

- C and C++ don't provide interfaces for file encoding detection and it isn't easy to implement well.
- Slide 20: Character Count vs Byte Count
 - Character counts tend to be close to code unit count for many languages for text encoded in UTF-16.
 - It is not easy to obtain a count of characters.
 - Corentin asked when it is useful to count characters.
 - Michael responded that a number of cases exist and provided an example of a buffer for which the user is told how many more characters they can expect to type; Twitter is an example for which both characters and bytes are counted.
- Slide 21: Character Encodings (Incomplete List)
 - In C and C++, `char` doesn't have a strongly associated encoding.
 - PBrett asked how often the lack of a strongly associated encoding leads to defects.
 - Michael responded that it is not as much of a problem as it used to be, but that there are still many locale dependent "ANSI" encoded files to be found.
- Slide 22: RTL Text Detection
- Tom asked the group what stood out to them from the presentation.
 - PBrett noted that C++ doesn't make it easy to write programs that are locale insensitive internally but locale sensitive at program boundaries.
 - Michael noted that `gettext()` provides an example of how plural forms can be handled.
 - Jens observed that, with `std::format()`, we're still far away from providing proper localization support; it doesn't yet lead to the pit of success.
 - Tom noted that the possibility of extending `std::format()` creates opportunity.
 - Michael noted that formatting is often used for internal uses that don't require localization or translation.
 - Steve stated that the experiences reported closely match his experience at Bloomberg.
- NB comment processing.
 - NB comment processing was postponed due to lack of time.
- Tom reported that he would not be available for the previously scheduled 2022-10-26 meeting and suggested rescheduling meetings for 2022-10-19 and 2022-11-02 with the intent to focus on addressing NB comments in advance of the Kona meeting; there were no objections.

October 19th, 2022

Draft agenda:

- NB comment processing.

Attendees:

- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [US 2-029 3.35 \[defns.multibyte\] Give context for "execution character set"](#):
 - Steve presented the concern:
 - The definition of multibyte character refers to the locale dependent execution character set.
 - Changing this might be difficult, but removing the reference to "execution character set" might help.
 - Hubert stated that the use of "multibyte character" in the library wording is consistent with the definition.
 - Tom asked if Hubert is suggesting that this is not a defect.
 - Hubert responded affirmatively.
 - Steve stated that he would agree if the definition was in the library section.
 - Jens explained that there used to be a terms and definitions section in the library wording but that ISO required it to be merged with the section in the core wording back in the C++17 time frame.
 - Hubert noted that the only use of "multibyte character" is in the library wording.
 - Corentin responded that there are indirect uses of it via "multibyte string" and "NTMBS" in the definition of the `main` function in [\[basic.start.main\]](#).
 - Corentin noted that all uses of it are intended to refer to the locale encoding.
 - Tom asked if it would make sense to strike the term so that it is inherited from the C standard.
 - Jens expressed a preference not to do so.
 - Steve stated that doing so might have unintended consequences.
 - Tom summarized the sentiment expressed so far; we're leaning towards this not being a defect but that there are opportunities for improvement via editorial changes.
 - Tom suggested that any such editorial changes be left up to the CWG.
 - Jens replied that the CWG is likely to decline to make any changes without a proposed change.
 - Corentin asked if an editorial pull request could be submitted.
 - Jens replied affirmatively.

- Hubert stated that the concern that Corentin raised regarding use of "multibyte character" with `main` is an issue.
- Jens asserted that would be a different core issue.
- **Poll 1: [US 2-029] SG16 suggests to consider this issue as "not a defect", but to improve the presentation by editorially moving the definition of "multibyte character" to [\[multibyte.strings\]](#).**
 - **Attendees: 8**
 - **No objection to unanimous consent.**
- [*Editor's note: Corentin submitted a pull request that implements the polled direction at <https://github.com/cplusplus/draft/pull/5910>.]*
- [US 38-098 22.14.6.4p1 \[format.string.escaped\] Escaping for debugging and logging](#):
 - Hubert presented the concern:
 - The feature description claims to provide a larger scope than it serves; the design doesn't suffice to address all logging scenarios.
 - It is not clear that the escaped string is required to be usable as a string literal.
 - Victor opined that the proposed change to replace "logging" with "technical logging" makes sense.
 - Victor expressed a preference against the second bullet regarding visually distinguishing equivalent text that is differently encoded.
 - Victor stated that the primary motivation for the feature was to produce a character sequence that would not interfere with the formatting of ranges.
 - Victor noted that the feature has existing experience with both Python and Rust and that the chosen design is modeled after Rust.
 - Victor asserted that the proposed change to allow for future addition of alternative escaping methods is unnecessary since other extension methods are already available.
 - Jens stated that the concern seems mostly related to the first sentence of [\[format.string.escaped\]](#):

A character or string can be formatted as *escaped* to make it more suitable for debugging or for logging.

- Jens continued; and the request is to make it clear that the escaped result shall be valid for interpretation as a string literal and that "logging" be replaced with "technical logging".
- Hubert agreed, but noted there is still a question of whether visually distinct output is desired.
- Hubert reiterated; the first priority is that the escaped result is a valid string literal, and a secondary priority is that text that might not be visually distinct be made so.
- Jens stated that the minimal change would be to change that first sentence.
- Jens noted that no actual defect has been identified.
- Hubert stated that SG16 may not be the best place to fully resolve the comment; the question of extension remains and is more of a LEWG consideration.

- Jens suggested that, for LWG's benefit, SG16 should propose a change to that first sentence.
- Corentin stated that NB comment FR-005-134 similarly states that the intent of the feature is not clear.
- Corentin asserted there are further questions regarding the escaping of grapheme clusters and that it is not clear what is intended to be escaped and for what purpose.
- Corentin expressed concern that the currently specified behavior of escaping all combining characters disadvantages some languages more than others and provided Korean as an example.
- Victor acknowledged that US 38-098 and FR-005-134 both state that the intent is not clear, but noted that their proposed resolutions are not in agreement.
- Victor agreed with Corentin that users of scripts that require more use of combining characters should not be penalized.
- Victor stated that the Python form of the feature does not escape combining characters and that can result in interference with range separators.
- Victor noted that the original proposal only escaped lone combining characters and acknowledged that the switch to the Rust approach might have gone too far.
- Hubert disagreed with the notion that a failure to escape does not harm the technical debugging use case.
- Mark reported experience with use cases where text content is only available via an image; perhaps a screen shot captured with a phone.
- Mark stated that he has only experienced a need for escaped characters in cases where the text was not correctly encoded.
- Mark noted it is a valid question as to whether the standard library should default to producing visually indistinct text.
- Tom stated that a goal of maximizing visual distinction would require escaping all characters not in the basic character set.
- Corentin replied that it would be terrible to escape all non-ASCII characters but that doing so would not be worse than escaping all combining characters.
- Corentin expressed a preference towards either maximizing escaping or minimizing it.
- Hubert stated that the scripts like Korean provide strong motivation for a minimally escaped form.
- Hubert noted that there are still valid reasons for wanting a visually distinct form via an easy opt-in.
- Victor reiterated that the primary goal of the escaped form was to avoid interference with the formatted range output.
- Victor suggested that desires for other use cases be pursued via new papers rather than NB comments.
- Victor asserted that it is useful to have ill-formed code units escaped.
- Mark expressed a preference towards not escaping combining characters due to the readability harm it would impose on scripts like Korean.

- Corentin asserted that all use cases can't be satisfied with this single facility but that extensions can satisfy more post-C++23.
- Corentin expressed a preference towards a default that maintains readability for more languages and that more extensive escaping can be pursued separately.
- Hubert opined that a sequence of combining characters that immediately follow an escaped character is sufficient evidence of an error to justify escaping them.
- Corentin repeated that it is useful to escape non-printable characters.
- Corentin stated that the grapheme breaking algorithm is potentially expensive, but then backtracked with an observation that it is sufficient to check for the `Grapheme_Extend=Yes` character property to identify combining characters that may need to be escaped.
- **Poll 2.1: [US 38-098] SG16 agrees that the formatted code units in the escaped string are intended to be usable as a string literal that reproduces the input.**
 - Attendees: 8
 - No objection to unanimous consent.
- **Poll 2.2: [US 38-098] SG16 agrees that the escaped string is intended to be readable for its textual content in any Unicode script.**
 - Attendees: 8
 - No objection to unanimous consent.
- **Poll 2.3: [US 38-098] SG16 agrees that separators and non-printable characters ([\[format.string.escaped\]p\(2.2.1.2\)](#)) shall be escaped in the escaped string.**
 - Attendees: 8
 - No objection to unanimous consent.
- **Poll 2.4: [US 38-098] SG16 agrees that combining code points shall not be escaped unless there is no leading code point or the previous character was escaped.**
 - Attendees: 8
 - No objection to unanimous consent.
- Tom stated that he would provide examples for each of the polls when reporting the SG16 consensus once the NB comment github repository is populated.
- Tom suggested that anyone that works on proposed wording include examples.
- [US 64-132 Annex E.4 Whitespace and pattern rules:](#)
 - Tom noted that FR-009-024, if adopted, will make this NB comment moot.
 - Corentin explained the motivation for the FR-009-024 comment; that the annex is light on information, that many of the requirements don't apply to C++, and that the ones that do could be noted in [\[lex.name\]](#).
 - Steve responded that an explicit record of a negative answer to a question is useful.
 - Steve explained that it would be difficult to identify Unicode requirement conformance information if it was spread throughout the standard wording.
 - Tom observed that differing opinions are clearly present with regard to the utility of the annex and stated that, due to time constraints, discussion will be limited to US 64-132 for now; discussion of FR-009-024 will be scheduled for a future meeting.

- Steve expressed an expectation of agreement that UAX #31 is intended to apply to general purpose programming languages.
- Hubert expressed a desire for more details and noted that conformance is not currently claimed.
- Tom provided a link to Unicode document [L2/22-179](#); it contains highlighted markup of the changes that were accepted for Unicode 15.
- Tom noted the changes added to the beginning of chapter 4, "Pattern Syntax":

Most programming languages have a concept of whitespace as part of their lexical structure, as well as some set of characters that are disallowed in identifiers but have syntactic use, such as arithmetic operators. Beyond general programming languages, there are also ...

and the changes to the "Modifications" section at the end of the document:

- Section 4, Pattern Syntax
 - Clarified that this section is applicable to programming languages.

- Jens observed that the NB comment is missing a reference to the updated Unicode document that clarifies applicability to general purpose programming languages.
- Jens suggested that the annex could state that [\[lex.name\]](#) defines a profile.
- Hubert expressed a preference to continue claiming non-conformance pending a clear specification of a conforming profile.
- Steve expressed contentedness with a change to just claim non-conformance.
- Jens observed that consensus appeared to be aligning with the proposed change from the NB comment as opposed to the one proposed in [P2653R0 \(Update Annex E based on Unicode 15.0 UAX 31\)](#).
- Steve asked if such a change could be applied editorially.
- Tom opined that it could be.
- Jens expressed a desire for CWG to review first and stated that, if a paper revision can be made available quickly, that he would schedule it for CWG review later in the week.
- Steve agreed to prepare a revision.
- **Poll 3: [US 64-132] SG16 agrees with resolving the issue in the direction presented in the comment.**
 - **Attendees: 8**
 - **No objection to unanimous consent.**
- Tom discussed plans for the next SG16 meeting:
 - Review of the GB and FR draft NB comments identified 7 comments for SG16 to review
 - It is not yet known if additional NB comments from other NBs will require review.
 - The next meeting is scheduled for 2022-11-02.
 - Once an agenda is sent, please discuss in email in advance of the meeting in order to reduce review time during the meeting.

November 2nd, 2022

Draft agenda:

- NB comment processing.

Attendees:

- Charles Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Mark de Wever
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [FR 005-134 22.14.6.4 \[format.string.escaped\] Aggressive escaping](#):
 - Corentin explained that the direction polled for [US 38-098](#) during the [October 19th, 2022 SG16 meeting](#) suffices to resolve this issue.
 - Victor agreed that the prior poll result is consistent with the first option of the proposed change.
 - **Poll 1: [FR 005-134]: SG16 recommends accepting the comment in the direction presented in the first bullet of the proposed change and as recommended in the polls for US 38-098.**
 - **Attendees: 8**
 - **Unanimous consent**
 - Corentin asked if anyone is preparing wording for US 38-098.
 - Hubert replied that no wording was provided with the comment.
 - Victor noted that the proposed change for US 38-098 did have the suggestion to replace "logging" with "technical logging".
 - Hubert replied that the direction polled didn't include that change.
 - Tom stated that wording will be left up to LWG without a volunteer.
- [GB-031 5.2 \[lex.phases\] Clarification of wording on new-line and whitespace](#):
 - Tom lamented Peter Brett's absence.
 - Tom recalled recent [discussion on the SG16 mailing list](#) that suggested a possible misunderstanding regarding feedback provided during the [2022-09-09 CWG review](#) of a draft of [P2348R3](#).

- Corentin explained that CWG was dissatisfied with the amount of churn involved in the paper and preferred an approach that addresses whitespace issues during translation phase 1.
- Corentin expressed disagreement with that approach and stated that he doesn't plan to pursue it.
- Corentin acknowledged that an issue exists.
- Steve expressed support for fixing the issue eventually but that he is weakly against doing so via an NB comment since, though the risk is low, late fixes can have unintended consequences.
- Jens disagreed with Corentin's summary of the CWG review, specifically with the claim that CWG wanted all whitespace issues to be addressed in translation phase 1.
- Jens explained that what CWG requested was for translation phase 1 to translate all accepted new-line forms to a single new-line character in the translation character set.
- Jens reported that CWG determined that the form of a new-line expressed in an input file is not observable by a program, not even in a raw string literal.
- Jens agreed with Corentin's claim that CWG recommended against the churn proposed in the paper.
- Jens explained the status quo, that translation phase 1 does not currently allow a UTF-8 encoded input file to have a new-line sequence other than U+000A (LINE FEED); the wording prohibits the use of U+000D (CARRIAGE RETURN) followed by U+000A (LINE FEED) as a new-line indicator.
- Jens noted that [US 3-030](#) requests a change that matches the CWG feedback.
- Tom asked Corentin if Jens' explanation was helpful.
- Corentin replied that he doesn't want to object to progress but that he lacks bandwidth to work on the issue himself.
- Corentin offered to share the source to [P2348](#) to anyone interested in working on a revision.
- Steve stated that, based on the intended scope, he would not object to CWG's preferred direction.
- Steve volunteered to look into producing a revision of P2348 if Corentin makes the source available.
- Tom stated that Jens' comments suggest a path forward of rejecting this comment in favor of pursuing US 3-030.
- Jens suggested that further action await a revision of P2348 and that this NB comment be handled procedurally as not having consensus for a change.
- Corentin noted that P2348 went through the committee pipeline and doesn't need to be rushed.
- [FR-009-024 Annex E \[uaxid\] Shorten contents and integrate with \[lex.name\]](#):
 - Tom mentioned that this issue was briefly discussed with the discussion of [US 64-132](#) during the [October 19th, 2022 SG16 meeting](#).

- Corentin stated that it is not yet clear that we understand [UAX #31](#) sufficiently well to declare conformance.
- Corentin asserted that, assuming retaining annex E is desirable, additional work is needed to evaluate conformance against a specific version of UAX #31, but it isn't clear which version that evaluation should be performed against.
- Corentin claimed that it is not clear that annex E is necessary or useful.
- Corentin noted that it would be useful to note some of the associations in [\[lex.name\]](#).
- Steve replied that the burden of conformance is the same regardless of where it is stated.
- Steve added that he is disinclined to abandon attempting to state conformance.
- Steve asserted that, similarly to undefined behavior, it is hard to find answers for things that are not explicitly stated in the standard.
- Steve claimed that statements regarding what is and is not intended to be conforming are useful.
- Steve noted that the placement of the conformance statements in an annex avoids interactions with normative wording.
- Jens reported that the reference to UAX #31 in the [bibliography](#) specifically refers to revision 33 and Unicode 13.
- Jens asserted that it is preferable that the C++ standard specify the syntax of identifiers itself rather than by deference to Unicode.
- Jens expressed support for expanding annex E to include statements of conformance for other Unicode requirements in a future standard.
- Jens noted that some of the clarifications made to UAX #31 for Unicode 15 were directly inspired by the initial attempt to state conformance in annex E and that such a feedback cycle is a valuable result.
- Jens stated that annex E doesn't require significant maintenance and, since it is non-normative, a failure to update it would not be highly consequential since it has no implementation impact.
- Corentin stated that the Unicode standard is defined as a complete set and is not intended or designed to support cherry picking different versions of its parts.
- Corentin provided normalization as an example of Unicode specification that is defined across multiple parts of the Unicode Standard.
- **Poll 2: [FR-009-024]: SG16 recommends rejecting the comment on the basis that explicit indication of Unicode requirement conformance, non-conformance, or inapplicability is useful.**
 - **Attendees: 9 (1 abstention)**
 - **SF F N A SA**
3 3 1 0 1
 - **Consensus.**
- [FR-010-133 \[Bibliography\] Unify references to Unicode](#) and [FR-021-013 5.3p5.2 \[lex.charset\] Codepoint names in identifiers](#):
 - Corentin explained that the C++ standard currently references four distinct Unicode versions for various purposes but that implementations, Clang specifically, intend to

adopt behaviors from newer Unicode versions as releases occur.

- Corentin described a technical inconsistency that results from the disjoint version references:
 - The range of UCS scalar values that can be expressed in a *universal-character-name* (UCN) is determined by the ISO/IEC 10646 version.
 - The set of character names recognized for a *named-universal-character* (NUC) are likewise determined by the ISO/IEC 10646 version.
 - The set of UCS scalar values allowed in an *identifier* is determined by the `XID_Start` and `XID_Continue` properties defined in the referenced [UAX #44](#) version.
 - If the version of UAX #44 referenced corresponds to a newer version of the Unicode Standard than the associated version for the referenced version of ISO/IEC 10646, then there will exist some identifiers that can be spelled as, for example, `x\u1234` but not as `x\N{NAME_FOR_1234}`.
- Steve expressed concern that updating the referenced versions might break section references.
- Corentin replied that he checked all references and only found one section reference; the reference for the Unicode replacement character in [\[ostream.formatted.print\]](#) specifically references chapter 3.9 of the core specification for Unicode 14.
- Steve stated that the bibliography is intended to reflect what the author was reading when writing the C++ standard.
- Corentin agreed and noted that normative changes should be made as necessary when the versions referenced in the bibliography are updated.
- Steve noted that such concerns will be more important for future library features that have a deeper dependence on the Unicode Standard.
- Tom noted that the ISO requires references to other ISO standards to reference the most recent version and asked if that applies to non-ISO standards as well.
- Jens replied that the ISO prefers undated references.
- Jens explained that outdated ISO versions don't really exist from the ISO perspective since a newer version is intended to replace a previous version; references to previous releases are somewhat like dangling pointers.
- Jens noted that, practically speaking, older versions do exist and that we do refer to older versions when necessary; like we have to do for UCS-2.
- Jens further explained that a dated reference is used for C since normative changes are very likely required to accommodate a newer version.
- Corentin explained that, at present, there is a mix of specific version references and floating references and that some are normative and some are non-normative.
- Corentin stated that the only change that would have a normative impact is for named character sequences.
- Jens stated that the Unicode Standard is referenced for cases where the needed subject matter is not present in an ISO standard.
- Jens noted that ISO prefers referencing ISO standards when possible.

- Jens suggested that the project editor should have more insight into the rules provided by ISO regarding references to ISO standards vs the Unicode Standard.
- Corentin clarified that the NB comment is not asking to only refer to the Unicode Standard; it is asking that named character sequences be made consistent with other uses of Unicode functionality.
- Jens noted that the character names are present in ISO/IEC 10646, but that the properties needed for identifiers are not.
- Hubert suggested that, when a reference is needed to the Unicode Standard, that the version aligned with ISO/IEC 10646 be referenced.
- Hubert stated that implementations can then veto that in favor of newer versions and that no one would complain.
- Hubert raised the option of asking the project editor to make a request to the ISO that the scope of ISO/IEC 10646 be expanded to include the additional Unicode features that we need.
- Hubert expressed a preference towards referencing ISO/IEC 10646 for terms and definitions because the ISO's practice tends to be more stringent than the Unicode Consortium's.
- Corentin repeated his goal to improve consistency; that the references be updated so that the character names and XID properties be sourced from the same reference.
- Hubert asked why the reference for extended grapheme cluster is non-normative.
- Jens replied that he thinks [UAX #29](#) is only referenced to satisfy normative encouragement for an implementation direction.
- Charlie expressed agreement with Jens' recollection.
- Hubert stated that normative encouragement should require a normative reference.
- Jens agreed that is probably true.
- Corentin asserted that, as more support for Unicode is added to C++, there will be more need for references to the Unicode Standard that can't be satisfied by ISO/IEC 10646.
- Jens admitted he was surprised when he first joined SG16 to learn that ISO/IEC 10646 specifies a subset of the features present in the Unicode Standard.
- Tom asked if changes to reference the Unicode Standard version that is aligned with the referenced ISO/IEC 10646 version would resolve the concern.
- Jens noted that the current reference to ISO/IEC 10646 is undated.
- MarkZ suggested the right approach would be to just reference the Unicode Standard.
- Corentin suggested that the next action be to coordinate with the project editor to better understand our options.
- Steve suggested it might be best to state that implementations should use the Unicode Standard version that aligns with their version of ISO/IEC 10646.
- Tom stated that the [Unicode FAQ](#) explicitly states which Unicode Standard version is aligned with each ISO/IEC 10646 version and asked if ISO/IEC 10646 is similarly explicit.

- Jens checked and reported that it is not, but that it embeds links that are version specific.
- Corentin stated that the highest priority is to provide consistent references and that we can rely on forward compatibility guarantees.
- Jens noted that, though we do understand and appreciate the Unicode stability guarantees, we are obligated to verify that those commitments are honored.
- **Poll 3: [FR-010-133][FR-021-013]: SG16 requests that the project editor discuss with the ISO the option of eschewing references to ISO/IEC 10646 in favor of the Unicode Standard both for technical consistency and release frequency.**
 - **Attendees: 9 (1 abstention)**
 - **Objection to unanimous consent.**
 - **SF F N A SA**
3 3 0 1 1
 - **Weak consensus**
 - **SA: Use of the ISO/IEC 10646 document benefits from ISO governance.**
 - **SA: Would prefer to explore expansion of ISO/IEC 10646 to include more components of Unicode.**
- Hubert indicated he might work with his NB to raise comments on the next ballot of ISO/IEC 10646 to request that it expand its scope.
- MarkZ suggested that quality issues could also be reported to the Unicode Consortium.
- MarkZ noted that interoperability with other languages and runtimes might be improved by aligning with the Unicode Standard.
- **Poll 4: [FR-010-133][FR-021-013]: SG16 recommends resolving these comments by restricting all references to the Unicode Standard to the version that corresponds to the referenced version of ISO/IEC 10646.**
 - **Attendees: 9 (1 abstention)**
 - **SF F N A SA**
2 3 0 3 0
 - **No consensus.**
 - **A: It doesn't benefit the community to reference a Unicode version that is outdated by the time the standard is published.**
- Steve suggested that it might be helpful to explore different guarantees for core language vs the standard library.
- Hubert agreed that it is conceivable that use of different Unicode Standard versions for the core language and the standard library would be ok.
- Tom reported that the next meeting will take place on November 30th.

November 30th, 2022

Draft agenda:

- [P2713R0: Escaping improvements in std::format](#)
 - [US 38-098 22.14.6.4p1 \[format.string.escaped\] Escaping for debugging and logging](#)
 - [FR 005-134 22.14.6.4 \[format.string.escaped\] Aggressive escaping](#)
- [P2693R0: Formatting thread::id and stacktrace](#)
 - [FR-008-011 22.14 \[format\] Support formatting of thread::id](#)
- [FR-010-133 \[Bibliography\] Unify references to Unicode](#) and [FR-021-013 5.3p5.2 \[lex.charset\] Codepoint names in identifiers](#)
- [P2675R0: LWG3780: The Paper \(format's width estimation is too approximate and not forward compatible\)](#)
 - [LWG #3780: format's width estimation is too approximate and not forward compatible](#)
 - [FR-007-012 22.14.2.2 \[format.string.std\] codepoints with width 2](#)
- [FR-020-014 5.3 \[lex.charset\] Replace "translation character set" by "Unicode"](#)

Attendees:

Charles Barto Corentin Jabot Jens Maurer Mark de Wever Mark Zeren Nathan Owen Peter Brett Tom Honermann Victor Zverovich Zach Laine

Meeting summary:

- [P2713R0: Escaping improvements in std::format](#):
 - Tom reported that the paper implements the previous guidance provided for [US 38-098](#) during the [2022-10-19 SG16 telecon](#) and for [FR 005-134](#) during the [2022-11-02 SG16 telecon](#) so all that should be needed is to confirm the paper via a poll.
 - Tom noted that some minor wording feedback was provided in a [post to the SG16 mailing list](#).
 - Victor presented the paper and further wording review commenced.
 - **Poll 1: P2713R0: Forward to LEWG as the recommended resolution of US 38-098 and FR 005-134 amended with discussed wording changes.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- [P2693R0: Formatting thread::id and stacktrace](#):
 - Corentin provided an introduction.
 - Victor reported Bryce's rationale for SG16 review; there were questions about wide string support.
 - Victor noted that the ostream inserters for stacktrace_entry and basic_stacktrace do not support wide ostreams, so the lack of support for std::format is consistent.
 - Corentin stated that there is no guarantee that std::thread::id will be formatted consistently for char and wchar_t.
 - Jens, referring to the proposed [stacktrace.format] wording, noted that "must" is not allowed in normative wording.
 - Victor asked what should be used instead.

- Tom suggested "mandates" or "requires".
- Victor explained that the wording intent is that a non-empty *format-spec* evaluated at compile-time render the program ill-formed and result in a format error exception if evaluated at run-time.
- Jens suggested wording the requirements in terms of format string validity.
- Charles noted that a thread ID is a handle on Windows.
- Charles stated that his only concern is whether additional header inclusion might be required but the proposal looks fine otherwise.
- Jens suggested dropping the "The syntax of format specifications is as follows" sentence in the wording for `formatter<thread::id, charT>`.
- Tom stated that any changes to require wide character support for `std::thread::id` would be out of scope.
- **Poll 2: P2693R0: Forward to LEWG as the recommended resolution of FR-008-011.**
 - **Attendees: 10**
 - **No objection to unanimous consent.**
- [FR-010-133 \[Bibliography\] Unify references to Unicode](#) and [FR-021-013 5.3p5.2 \[lex.charset\] Codepoint names in identifiers](#):
 - Corentin explained that authoring a paper to address these NB comments is on his todo list.
 - Corentin invited offers to help with a paper.
 - Jens stated that it will be important to understand how the change to the normative reference impacts how wording is interpreted throughout the standard.
- [P2675R0: LWG3780: The Paper \(format's width estimation is too approximate and not forward compatible\)](#):
 - Corentin provided an introduction.
 - Victor had initially identified a range of code points that specify characters to be considered as having an estimated width of two.
 - That code point range corresponds to Unicode 13 and has not been updated for more recent Unicode Standard versions.
 - Analysis of source code and behavior in existing terminals inspired the current proposal to derive the code point ranges from the Unicode character property database.
 - Victor expressed mixed feelings regarding the proposal; though the idea is favorable, consulted sources indicate that the Unicode properties don't predict how characters are displayed particularly well.
 - Victor indicated support for consideration of the Unicode width property, but that code point ranges that are ambiguous should be retained.
 - Victor stated that all of the code points that change from an estimated width of two to an estimated width of one are rendered with a width of two in his environment, so those cases appear to constitute a regression.
 - Victor acknowledged that the proposal looks like a good step in the right direction.

- Victor raised U+2E9A as an example; it is an unassigned character in a block for which characters are assumed to have a width of two and it is rendered as a wide unassigned character.
- [*Editor's note: In Unicode 15.0, [U+2E9A](#) is a reserved unassigned character in the [CJK Radicals Supplement block](#) and its `East_Asian_Width` property value is `N` (Neutral).*]
- Corentin replied that terminals that display such characters as wide characters are non-conforming.
- Corentin argued that use of the Unicode character database is justified by the lack of anything obviously better.
- Corentin asserted that estimated width is necessarily an approximation at present.
- Corentin stated his goal with the proposal is to prioritize a principled solution with predictability.
- Zach observed that there appear to be some contradictions and pondered how they might be resolved.
 - There is a desire to be forward compatible and to defer to the Unicode Standard.
 - There is a desire to consider certain unassigned code points as wide until they are assigned a width by the Unicode Standard.
- Corentin stated that existing behavior should be evaluated before choosing to deviate from Unicode.
- PBrett observed that wide divergence can be observed between different rasterizers and stated that he does not relish the idea of identifying the subset of behavior that is exhibited in the wild.
- Victor expressed skepticism regarding the feasibility of relying only on Unicode.
- Victor stated that Unicode conformance doesn't apply to this situation.
- Victor cautioned that the traditional Windows console behavior should not be used as a reference as it exhibits notoriously poor behavior.
- Corentin indicated an intent to update the paper with references to the scripts used to collect data and evaluate behavior.
- [FR-020-014 5.3 \[lex.charset\] Replace "translation character set" by "Unicode"](#):
 - Discussion was postponed due to lack of time.
- Tom reported that the next meeting will take place on December 14th, 2022.

December 14th, 2022

Draft agenda:

- [D2675R1: LWG3780: The Paper \(format's width estimation is too approximate and not forward compatible\)](#)
 - [LWG #3780: format's width estimation is too approximate and not forward compatible](#)

- [FR-007-012 22.14.2.2 \[format.string.std\] codepoints with width 2](#)
- [FR-020-014 5.3 \[lex.charset\] Replace "translation character set" by "Unicode"](#)

Attendees:

- Charlie Barto
- Corentin Jabot
- Jens Maurer
- Mark de Wever
- Peter Brett
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [D2675R1: LWG3780: The Paper \(format's width estimation is too approximate and not forward compatible\)](#):
 - *[Editor's note: D2675R1 was the active paper under discussion at the telecon. The agenda and links used here reference P2675R1 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - PBrett summarized the changes in the draft R1 revision.
 - Corentin summarized an [email sent by Victor](#) that demonstrated behavior in which a wide character was rendered such that it overlapped an adjacent character because the terminal treated the character as a narrow one but the font in use rendered it as a wide character.
 - Corentin pointed out that the demonstrated behavior implies that character width cannot be determined by looking at a rendered character in isolation since the character rendering may exceed the bounds of a terminal cell.
 - Victor acknowledged it was a mistake to categorize the relevant characters as having a width of 2; the initial error was due to observing the rendered character without an adjacent character.
 - Victor expressed appreciation for the systematic approach proposed in the paper and that it appears to improve behavior.
 - Victor stated that it is difficult to interpret the screenshots currently in the paper.
 - PBrett suggested that it might be helpful to provide more constructive feedback to paper authors regarding how presentation can be improved.
 - Corentin explained that he had asked for contributions of screenshots from others since he did not have convenient access to the wide range of terminals that are used in practice.
 - Corentin reported that rendering issues that occur with just one or a small subset of terminals are common and asserted that we should not concern ourselves with such cases.

- Corentin stated that he has not found cases that are contrary to the proposal and that have consistent behavior across the sampled terminals.
- Victor, referring to an [email that Tom sent to the SG16 mailing list](#), reported having performed some further analysis with the attached source code and provided some constructive feedback.
- *[Editor's note: The mailing list software appears to have ignored, misplaced, or otherwise omitted the source code that was attached to that email.]*
- Tom stated that we could spend additional time discussing the pros and cons of the screenshots but that doing so might not be a good use of our time.
- Corentin opined that it would not be a good use of our time and agreed to remove most of the screenshots.
- Jens summarized his understanding of the paper; that the standard currently specifies explicit code point ranges and the paper proposes changes to better align behavior with various terminals.
- Tom voiced agreement.
- Jens expressed concern that it is late in the release cycle for such changes.
- PBrett replied that this addresses a defect.
- Corentin noted that [LWG issue 3780](#) already exists.
- Tom explained that we can choose between recommending this as a change for C++23 or as a DR to be addressed in C++26.
- PBrett expressed a preference for addressing this in C++23.
- Victor noted that there already is consensus that width estimation is best effort and likely to change in the future.
- Victor stated that there is not an urgent need to rush this into C++23 but that we might as well add it now if we agree the paper is ready.
- Corentin explained that his motivation for targeting C++23 is to ensure that behavior varies as expected with whatever Unicode version is in use by an implementation.
- Corentin noted that the situation will grow worse over time as the explicit code point ranges in the standard deviate further from existing practice as that practice changes with new Unicode releases.
- **Poll 0.1: Forward D2675R1 "format's width estimation is too approximate and not forward compatible", with improved presentation, to LEWG as the recommended resolution of LWG3780 and NB comment FR-007-012.**
 - Attendees: 6
 - SF F N A SA
3 3 0 0 0
 - Unanimous consent.
- **Poll 0.2: Recommend that D2675R1 be applied to the C++23 working paper.**
 - Attendees: 6
 - SF F N A SA
2 4 0 0 0
 - Unanimous consent.

- [FR-020-014 5.3 \[lex.charset\] Replace "translation character set" by "Unicode"](#):

- Tom asked what new information has become available since we last discussed and polled this topic during the [2021-03-24 SG16 meeting](#).
- PBrett responded that the existence of an NB comment may constitute new information.
- Corentin stated that removal of the "translation character set" term will require addressing the imprecise use of the term "character".
- Corentin reported that the Unicode Standard states that an unassigned character must not be treated as a character and that treating one as such could be a Unicode conformance concern.
- Corentin requested an indication of support for this direction before devoting the considerable time drafting a paper would require.
- Jens noted that we don't claim conformance with the Unicode Standard; we only use it as a reference.
- Tom opined that the current use of "character" does not constitute a Unicode conformance concern.
- Tom asserted that a paper to address the imprecise use of "character" would be quite valuable regardless of any changes with respect to "translation character set".
- PBrett expressed support for making changes with regard to "translation character set" either in C++23 or sometime after the use of "character" is addressed.
- Corentin noted that the Unicode Standard intentionally does not define "character".
- Corentin indicated that the paper he would write would address the core language, but not the standard library since addressing both would require such a significant effort.
- PBrett asked if these changes could be done editorially.
- Jens replied that there is potential for friction with the C standard since it also uses the term "character".
- Tom reported that Ken Whistler recommended reviewing [UTR #17 \(Unicode Character Encoding Model\)](#) for terminology to use.
- Corentin replied that he would review it.
- Corentin noted that, after translation phase 1, the elements of the translation character set are all Unicode scalar values because surrogate code points are not allowed and asked what terminology should be used.
- Tom replied that, in an offline discussion, he had suggested to Corentin that we prefer "code point" in general discussion and reserve "scalar value" for use as a form of qualifier to restrict code point allowances.
- Jens requested a paper that describes the desired end state before considerable effort is put into producing wording.
- PBrett replied that doing so implies rejecting the NB comment.
- Jens replied that, without a paper, rejection is the only option as there can be no consensus for a specific change.
- Tom noted that there is very little time left for making changes to C++23.
- **Poll 1.1: Encourage further work on expressing the semantics of C++ lexing in terms of the terminology defined in the Unicode Standard.**

- **Attendees: 6**

- **S F F N A S A**

- 4 1 0 1 0**

- **Strong consensus.**

- **A: I'm concerned about interaction with the C standard and introducing inconsistency between core wording and library wording.**

- [D2736R0: Referencing the Unicode Standard:](#)

- *[Editor's note: D2736R0 was the active paper under discussion at the telecon. The agenda and links used here reference P2736R0 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
- Corentin noted that the previous feedback was to try to ensure that the change of reference would have no normative impact on behavior.
- Corentin explained that there is a design question regarding the `__STDC_ISO_10646__` predefined macro; the macro is specified by the C standard as having a value that reflects the date of a ISO/IEC 10646 standard.
- Corentin reported that there are known issues with the macro; compilers can't predefine it because the value to define it to is determined by the C standard library.
- Corentin stated that the macro is only useful to distinguish between old 16-bit Unicode and modern 21-bit Unicode.
- Corentin suggested that the C++ standard could specify it to have an implementation-defined value like it does for `__STDC_VERSION__`.
- Corentin suggested another alternative would be to specify it as having a Unicode version date instead.
- PBrett suggested specifying it to have a value that matters.
- Corentin explained that implementations that use a 16-bit `wchar_t` can't define this macro to any relevant Unicode or ISO/IEC 10646 standard.
- Jens replied that in those cases, he would expect the macro to be defined for the last ISO/IEC 10646 standard that had a 16-bit code point space.
- Jens suggested the value should just reflect the size of `wchar_t`.
- Corentin noted that the macro also reflects whether values of `wchar_t` correspond to a Unicode encoding; which could be locale dependent.
- Tom summarized three possibilities:
 - `wchar_t` has an associated encoding that is not a Unicode encoding; the macro is not defined.
 - `wchar_t` is 16-bit and the associated encoding is UCS-2; the macro is defined to reflect an obsolete ISO/IEC 10646 standard.
 - `wchar_t` is 32-bit and the associated encoding is UTF-32; the macro is defined to reflect a relatively current ISO/IEC 10646 standard.
- Jens opined that this requires coordination with WG14.
- PBrett asked if we can deprecate the macro.
- Jens replied that we can choose to deviate from the C standard but noted that the macro can be useful.

- PBrett asked about Corentin's previous suggestion to just state that the macro has an implementation-defined value.
- Jens opined that the macro has some value.
- Jens noted that the C++ standard has library wording that states that all elements of the wide character set are representable as values of `wchar_t` and that the presence of the macro definition in core wording is suggestive of applicability to wide character and string literals.
- Tom suggested some compare and contrast analysis with the C standard.
- Corentin stated that it isn't clear to him that WG14 knows what this macro is intended for.
- Corentin pondered deprecation, but not as a part of this paper.
- Corentin reported that code searches revealed few references to the macro that are sensitive to the macro value; most code just checks if the macro is defined.
- Tom announced that the next two telecons are scheduled for 2023-01-11 and 2023-01-25 and will be followed by the WG21 meeting in Issaquah in early February.